czii-cryo-et-object-identification

10th place solution

Rank	10
Team	Josef Slavicek
Author	Josef Slavicek
Topic ID	561844
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561844
Retrieved with Kaggle CLI on 2026-10-21.

I wish to apologize to organizers: they gave us very well prepared interesting competition in the hope to promote substantial progress in CryoET processing. What they get from me is just ensemble of vintage 3DUNets.

And I wish to thank to authors of some helpful notebooks, especially this. Without them I would not be able to develop anything working.

Short description

My solution is ensemble of 9 3DUnets. All except of one were first pretrained on simulated data and then finetuned on competition train data. I ensemble them by just averaging equaly weighted logits. Then I compute probabilities, threshold them to obtain regions of detection and then I apply some postprocessing to split regions into individual particles. All my trainings were in fact Optuna hyperparam-searching sessions from which I chose best models according to held-out local validation set.

Validation

I made the train / validation split just by experiment ID. For finetuning on competition train data, I train on 6 volumes and validate on one remaining. For pretraining on simulated data I still use performance on one of train data experiments for validation.

Data processing, augmentations

The only data processing I use was normalization with monai.transforms.NormalizeIntensityd. Regarding augmentations, I use RandFlipd, RandRotated and RandRotate90d from monai.transforms. During training I use simple implementation of mixup but Optuna chose to give it very small `alpha` (~0.03).

NN architecture

All my nets are the same architecture: monai.networks.net.UNet with `spatial_dims=3, channels=(48, 64, 80, 80, 128), stride_patterns=(2,2,2,1)`.

Training

All my trainings were done as Optuna hyperparam-search session where the inner trainig was done in pytorch lightning. I was using AdamW optimizer with cosine LR scheduler. Optuna settled to some unusual values of Adams beta parameters (e.g. `beta1=0.7, beta2=0.9996`).

Training objective

For finetuning on competition data I was asking UNet to predict particle classes map. As a loss function I was using weighted combination of Tversky loss and multiclass crossentropy. I clipped input logits for Tversky loss. For pretraining on simulated data I used 3 versions of objective.

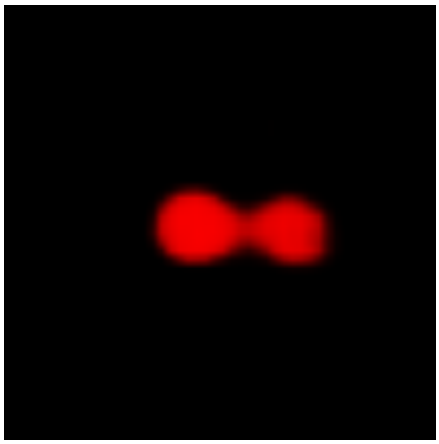
1) the same as fintuning.

2) with particle orientation vector as another objective. I hoped that it will force the network to understand particle shapes, but in fact the network didn't predict the orientation better than random guessing.

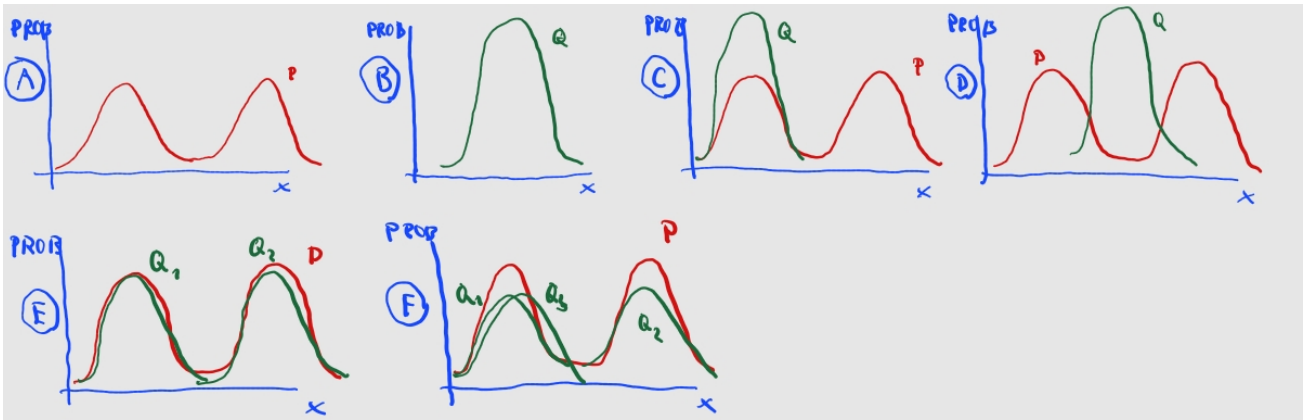
3) some my earlier implementation of 2) where there was bug making the orientation really random. Ironically, this was my most successful pretraining, present in 4 out of 9 my final networks

Postprocessing

After fusion, computing probabilities and thresholding, we get binary maps representing particles. Most of the time, isolated clusters of positive values really correspond to just single particle. But sometimes I have seen something like this:

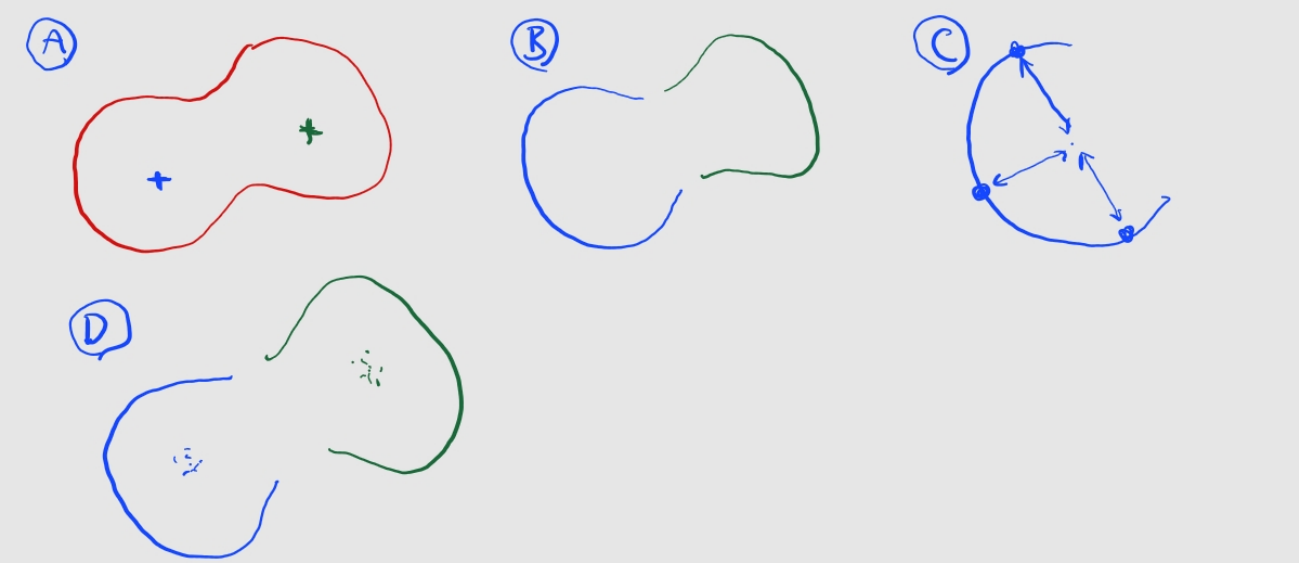


So we need to recognize and split such multiple detection. I guess there are some well established, proven solution how to do it, but I took it as a nice opportunity to reinvent wheel an I end up doing following (explained on 1D image for clarity):



We have probabilty density function (pdf) which is prediction of our model (image A) - let's name it P. We know very well how pdf of single particle (let's call it Q) looks like (image B), only we don't know where it is and how many of them is there. But if we place Q over P such way to minimize KL divergence, we get exactly what we want (image C). Only we must be careful because KL is not symmetric, so inserting P and Q in wrong order into KL formula would lead to undesirable result (image D). But if we are doing it correctly, we have placed first particle and if we place other, it will occupy other node of P (image E). We can continue doing this. Once our placed Qs start to overlap (image F), we are done.

So this is first part of my crazy KL machinery. To explain the remainder, let's switch to 2D view and forget meaning of colors from previous image - it will now be different:



So we have our model prediction, from which we can draw binary mask, from which we can detect border - red line in image A. And we know centroids of individual particle pdfs - green and blue cross. Just using these centroids would give us good prediction but I got some smallish improvement from following trickery. We can split the red border to parts belonging to individual particles (image B). Now let's process them separately. If we randomly pick 4 points (4 points in real 3D problem, while in our 2D image it would be 3 points) and find their circumcenter (unique point equally distant from them), we get good candidate for particle centroid (image C). So I use this to obtain clouds of particle center candidates and then I find the final centroid prediction by averaging some dense core of this cluster (image D).

That's it. Sorry. I couldn't resist :-)

This whole KL magic improve my LB score by ~0.015 and it takes 90min to run on P100.

Supplemental Q&A

> **Reza Paraan · 2025-02-19T15:44:57.847000**
>
> Hi Josef. Thanks again for participating and for your valuable report. I have a question regarding your post-processing. Do you calculate particle diameters after this step? If so, do you use that information to decide whether that particle belongs to the right class or not? For example, a lot of apoferritin particles cluster together. Without your postprocessing this cluster might seem like one big particle such as a VLP. I'm just curious if your method takes care of that kind of particle class confusion. Thank you so much.

> **Josef Slavicek · 2025-02-21T22:21:48.993000**
>
> Hello Reza, I apologize for the late reply, I was not monitoring this discussion.
> No, I process prediction of each class separately. Particle diameter enters this calculation as a fixed parameter and my postprocessing is trying to do just single thing: to place single particle predictions over the NN output for that class.